

1 · WHAT CASSANDRA IS — MASTERLESS, AP, AND OPTIMISED FOR WRITES

The definition

A **masterless, distributed, wide-column store**. Every node is identical — there is no primary, no coordinator role that must be elected, and no single point of failure. Data is distributed by consistent hashing over a token ring, and replicated to a configurable number of nodes per data centre.

The design goal, stated once: *linear write scalability and continuous availability across data centres*. Doubling the nodes roughly doubles write throughput, and losing a node — or a whole region — does not stop writes. Everything awkward about Cassandra is the price of those two properties.

The three properties that explain everything else

- Writes never read first. A write is an append to a commit log and a memtable — no lookup, no read-modify-write, no locking. This is why writes are fast and uniform, and it is why a delete is itself a write (a tombstone), which causes most operational trouble
 - There is no query planner and no join. You get exactly the queries your table was designed for. The **data model is designed per query**, and denormalisation is the intended practice rather than a compromise
 - Consistency is a **per-operation choice**. It is AP by default, and the same cluster behaves closer to CP when you ask for quorum on both sides. See panel 5
- Almost every Cassandra failure traces to one of those. Panels 2 and 9 cover the two that cause real incidents: a partition key chosen badly, and tombstones left unrepaired.

2 · DATA MODELLING — QUERY FIRST, AND THE PARTITION KEY DECISION

The governing rules, and I genuinely **invert relational practice**: start from the queries, not the entities. Write down every query the application will make, then design one table per query, duplicating data freely. Normalising first and hoping the queries work out produces a data model that cannot serve them at all.

The primary key has two parts, and they do different jobs

PRIMARY KEY (user_id, day, event_time, event_id)	CLUSTERING COLUMNS (device_id, reading_time)
PARTITION KEY OCCURS, WHICH PAGES STORES IT	CLUSTERING COLUMNS OCCUR, THE ORDER WITHIN THE PARTITION

Partition key

Heard to a disk, which determines the node. Every query must **efficiently** find it — without it there is no way to know which node holds the data

Sort rows on token within the partition. Range scans and ordering are efficient only in this order

Clustering columns

Sort rows on disk within the partition. Range scans and ordering are efficient only in this order

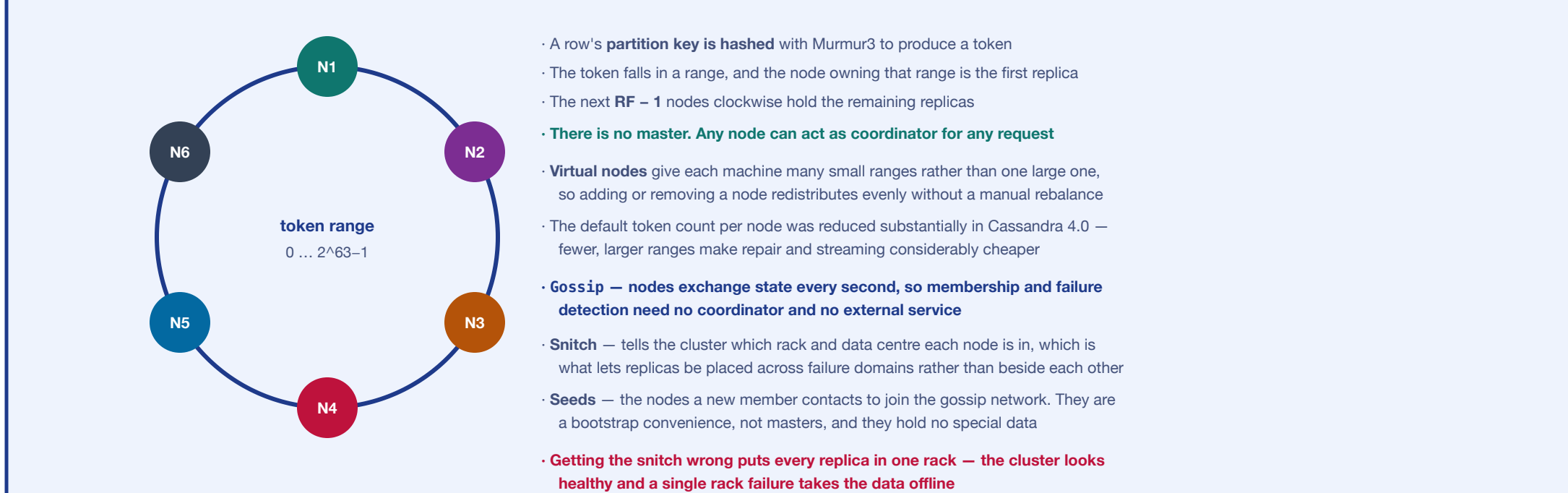
Static columns

One value shared by the whole partition

- Denormalisation is the design, not a shortcut
- The same data lives in several tables, each shaped for one query. This is correct and expected
 - The application maintains the copies. There is no join and no cascade — writing to all of them is your code's responsibility
 - Storage is cheap; a query that cannot be served is not

4 · ARCHITECTURE — THE TOKEN RING, REPLICATION, AND THE WRITE AND READ PATHS

THE TOKEN RING — every node is identical, and any node can serve any request



READING THE DIAGRAM — five properties that explain the rest of this sheet

- There is no master, and the partition key decides everything. Which node holds a row, whether a query can be served at all, and whether load spreads evenly are all consequences of that one choice — which is why panel 2 is the longest on this sheet.
 - Writes never read first, so a delete is a write. That makes writes fast and uniform, and it means deletion leaves tombstones that must be replicated, read through, and eventually purged — the source of most operational trouble.
 - Consistency is chosen per operation, and R + W - RF is the whole rule. The cluster is AP by default and behaves closer to CP when you ask it to. Inheriting the default means running at ONE, which is a choice whether or not anyone made it deliberately.
 - SSTables are immutable, so reads merge across files. Compaction exists to keep that count low, when it falls behind, read latency rises with no change to the queries. SSTables per read is the number that tells you.
 - Convergence is a maintenance activity, not an automatic one. Hints expire and read repair only touches data that is read. Anti-entropy repair is the only mechanism reaching every row, and it must complete within the grace period.
- Useful interpretation of any Cassandra design: what is the partition key, how large does one partition get at full retention, when did repair last finish, and what consistency level is the application actually using?

5 · CONSISTENCY LEVELS — WHAT EACH ONE ACTUALLY GUARANTEES

Level	Requires
ANY	The write succeeds even if no replica received it — a hint stored on any node counts. The weakest level that exists; the data is unrecoverable until the hint replays, and only once.
ONE	One replica responds. This is the default for reads and writes unless the driver or session overrides it.
LOCAL ONE	One replica in the local data centre. Commonly the driver default in multi-DC deployments.
TWO / THREE	That many replicas, anywhere.
QUORUM	A majority across all data centres. Correct but often unintended in a multi-DC cluster — it pays cross-region latency.
LOCAL_QUORUM	A majority within the local data centre. The production standard for multi-DC.
EACH_QUORUM	A quorum in every data centre. Write-side only; unavoidable if any DC is unreachable.
ALL	Every replica. Any single node down makes the operation fail — it trades away the availability you chose Cassandra for.
SERIAL	Read levels used with lightweight transactions, so a read observes in-progress Paxos state.
LOCAL_SERIAL	

Key cache

Caches whole rows. Cheap and effective — usually worth increasing

Row cache

Caches whole rows. Off by default and usually cannot be levered off — it is invalidated on any write to the partition, so it hurts mixed workloads

Rely on the operating system page cache. Cassandra reads SSTables through it, so unallocated memory is doing useful work. This is why heap should stay modest and the rest of RAM left free — raising the heap to consume the machine reduces the cache that makes reads fast.

7 · STORAGE ENGINE — SSTABLES, CACHES AND WHY READS COST MORE THAN WRITES

Cassandra is a log-structured merge store. Writes append to memory and are periodically flushed to immutable files. Nothing is ever modified in place — which makes writes sequential and cheap, and makes reads a merge across however many files hold fragments of the row.

The components

Commit log

Append-only, per node. The durability guarantee — a crash replays it. Put it on separate storage from data if the disks are not fast

Memtable

In-memory, sorted, per table. Absorbs writes until flushed

SSTable

Immutable sorted file on disk. Once written it is never modified — only merged away by compaction

Bloom filter

Per SSTable. Answers "this key is definitely not here" without a disk read, so most SSTables are skipped. False positives cost a wasted lookup, false negatives cannot occur

Partition index and summary

Locate a partition's offset within an SSTable

Key cache

Caches whole rows. Cheap and effective — usually worth increasing

Row cache

Caches whole rows. Off by default and usually cannot be levered off — it is invalidated on any write to the partition, so it hurts mixed workloads

Rely on the operating system page cache. Cassandra reads SSTables through it, so unallocated memory is doing useful work. This is why heap should stay modest and the rest of RAM left free — raising the heap to consume the machine reduces the cache that makes reads fast.

9 · TOMBSTONES & REPAIR

A delete is a write. It cannot modify existing SSTables because they are immutable, so it writes a **tombstone** — a marker saying "this is deleted as of this timestamp". The tombstone must be replicated everywhere, and it must survive the data it hides.

Why a read gets slower after deletes

- Tombstones are read like any other data. Scanning a partition means walking through them to determine what still exists
- Thresholds exist and they fire. Passing a certain count per query logs a warning; passing a much higher one aborts the query outright
- This is exactly why the queue pattern fails — the deleted head of the partition is scanned on every read, and grows

The critical rule, and the reason repair is mandatory. Tombstones are purged after **gc_grace_seconds** — ten days by default. If a node was down or missed the tombstone and it is purged elsewhere before that node learns of it, the deleted data comes back when the node rejoins. It is a real, silent correctness failure with a name: **zombie data**.

Therefore, **nodetool repair** must complete on every node with **gc_grace_seconds**. This is not a tuning suggestion — it is a correctness requirement of the delete mechanism. A cluster where repair has not run in two weeks with the default grace period may already have resurrected data, and nothing will report it.

Repair in practice

- Schedule it — with Reaper, or the built-in scheduling in newer versions. Repair that depends on someone remembering does not happen
- Subrange or incremental repair to keep each run manageable on large clusters
- It is expensive — it streams data and compares with live traffic. Run it continuously at low intensity rather than as an occasional large job
- Alert on time since last successful repair per node, against the grace period

Reducing **gc_grace_seconds** shortens the repair deadline proportionally. It is a legitimate choice for TTL-only tables where nothing is explicitly deleted — and dangerous if repair cannot reliably finish inside the shorter window.

12 · OPERATIONS — NODE LIFECYCLE, SCALING AND THE COMMANDS THAT MATTER

Adding and removing nodes

Adding a node

Bootstraps by streaming the ranges it will own from existing nodes. **Add one at a time** and let each finish — parallel bootstraps cause range confusion and heavy streaming load

After adding

The step most often forgotten. Existing nodes still hold data for ranges they no longer own. Until cleanup runs, **disk usage does not fall** and people conclude the system is broken

Removing a live node

decommission — it streams its data to the new owners before leaving. **Graceful**

Removing a dead node

removeNode — the cluster restores replica counts from surviving copies. Follow with **repair**

Replacing a node

Start the replacement with the replace-address flag so it assumes the dead node's ranges rather than joining as a new member

Scaling is linear and it is not instant. Bootstrapping streams real data over the network, and on a large cluster that is hours. **Plan capacity ahead of need** — adding nodes during an incident adds streaming load to an already struggling cluster.

Rolling restarts are routine and safe at RF 3 with quorum reads and writes — one node at a time, waiting for the cluster to report it fully up before continuing. Run **nodetool drain** before stopping a node so it flushes memtables and stops accepting writes cleanly.

14 · CONFIGURATION, DRIVERS AND OBSERVABILITY

Setting	Why
cluster_name	Nodes only join a cluster of the same name — the guard against a test node joining production
seeds	A few stable nodes per data centre for gossip bootstrapping. Not masters , and they should not all be in one rack
endpoint_snitch	Ones that direct traffic to the right data centre. It determines rack and data centre awareness, and therefore whether replicas are actually spread across failure domains
num_tokens	Virtual nodes per host. Modern defaults are far larger than early versions, which makes repair and streaming much cheaper. Keep it consistent across the cluster
commitlog_directory	Separate devices where possible, so commit log writes do not contend with compaction I/O
data_file_directories	Sized against cores and disks. Too high causes queuing and dropped messages rather than throughput
current_reads	Throttles compaction so it does not starve queries. Raise to clear a backlog
current_writes	Per table. Your repair deadline — see panel 9
compaction_throughput	Moderate heap, modern low-pause collector. Pause time matters more than size
gc_grace_seconds	
Heap and collector	

Host settings matter too. Disable swap entirely, raise file descriptor limits, disable transparent huge pages, use fast local disks, and run **NTP everywhere** — clock skew causes silent data loss under last-write-wins.

15 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

Symptom	Cause	Fix
Reads on one table time out, others fine	Tombstones — usually a queue pattern or heavy deletes	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Deleted data reappears	Repair did not complete within gc_grace_seconds	Schedule repair. Alert on time since last successful run per node
One node hot, others idle	Hot partition from a low-cardinality partition key	Add a bucket to widen the key. Requires a model change and a migration
Reads slow down gradually	Compaction behind, so SSTables per read is climbing	Check pending compactions; raise throughput; review the strategy
Disk usage did not fall after adding nodes	nodetool cleanup was never run	Run it on every pre-existing node after any topology change

16 · PRODUCTION FAILURES — SYMPTOM, CAUSE AND FIX

Symptom	Cause	Fix
Every query written down before the tables were designed	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
One table per query, denormalisation everywhere	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Every read supplies the full partition key	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
No partition can grow unbounded — bucketed where needed	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Partition sizes monitored, under roughly 100 MB	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Partition keys cardinality checked for hot spots	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Clustering order matches the required read order	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
No ALLOW FILTERING anywhere in application code	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
No queue-like or delete-heavy pattern	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
TTLs used in preference to explicit deletes	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
NetworkTopologyStrategy, never SimpleStrategy	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
RF 3 per data centre for anything that matters	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Strictly configured and replica placement verified across racks	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions
Consistency level set explicitly — the default is ONE	Writes are slow, reads are fast	Check tombstone counts in tablestats; redesign to TTL or bucketed partitions

The one that summarises the sheet: name the partition key for every table, state how large the largest partition becomes at full retention, say when repair last completed on every node, and give the consistency level the application actually uses. Those four answers cover the failures that genuinely take Cassandra down — and none of them are about throughput, which is the one thing it will do well regardless.

Where it sits on CAP

Cassandra is AP — during a partition, nodes on both sides keep accepting reads and writes at low consistency levels, and the cluster converges afterwards through read repair, hinted handoff and anti-entropy repair. Availability is preserved; a read may return stale data.

The nuance worth adding. Because consistency is tunable per operation, a write and a read that both use QUORUM give linearizable-per-key behaviour for that operation — at the cost of failing when a quorum is unreachable. Cassandra is not read at AP; the default AP and you choose the position per statement. That is the most accurate answer to the question.

Use it when

- Write volume is very high and continuous — event logs, clickstreams, sensor and telemetry data, time-series
- Always-on across regions is a genuine requirement, with no fallback window
- Access is by a known key and the queries are stable and few
- Linear scale-out is needed and adding nodes must not require re-architecting
- Eventual consistency is acceptable for most reads

Partition sizing is where reads are made or broken. A partition lives entirely on its replica nodes and is read as a unit. Keep partitions under roughly 100 MB and 10,000 rows as a working guide. Beyond that, reads slow, compaction struggles, repairs take longer, and memory pressure rises on exactly the nodes holding it.

The two ways a partition key goes wrong

Unbounded growth

The partition has no natural limit — all events for a device, forever. It grows until it breaks. The fix is bucketing: add a time component such as day or month to growth

Hot partitions

One key receives a disproportionate share of traffic — a status flag, a country, a popular tenant. Low cardinality is the warning sign. That partition's replicas are saturated while the rest of the cluster is idle, and adding nodes does not help

Unbounded: grows forever

PRIMARY KEY (device_id, reading_time)

Unbounded: one partition per device per day

PRIMARY KEY (device_id, day)

Not a handful of distinct values

PRIMARY KEY (status, order_id)

Unbounded: add a bucket to widen the key

PRIMARY KEY (status, bucket, order_id)

The trade with bucketing is query fan-out. Reading a month now means reading thirty partitions rather than one. That is usually the right trade — thirty bounded reads are far better than one unbounded partition — but choose the bucket size against the query, not arbitrarily.

The test to apply before committing to a key, how many distinct values does it have, how large can one partition become at full retention, and does every read supply RF 3 or any answer is uncomfortable, change the model now — it is by far the most expensive thing to change later.

REPLICATION — set per keyspace, per data centre

NetworkTopologyStrategy — use this, essentially always

Replication factor is declared per data centre, and replicas are placed on distinct racks where the topology allows it. RF 3 per data centre is the standard production choice: it tolerates one node loss per DC while still satisfying a local quorum.

SimpleStrategy ignores rack and data centre entirely. It is responsible for a silent and wrong for anything else — including a single-DC production failure, because migrating every row from 1 to 2 is a disaster.

THE ONE FORMULA TO REMEMBER — R + W - RF

If the number of replicas read plus the number written exceeds the replication factor, the two sets must overlap by at least one node — so the read is guaranteed to see the latest write. This single inequality is the whole of tunable consistency, and it is what "strong consistency" means here.

With RF 3, QUORUM is 2. With a QUORUM and read at QUORUM gives 2 + 2 - 3 = consistent, and tolerant of one node being down.

With a ONE and read at ONE gives 1 + 1 - 2, which is not greater than 2 — so the read may legitimately return stale data. Both are valid choices; only one of them is a consistency guarantee, and the difference should be deliberate rather than inherited from a default.

MULTI-DATA-CENTRE — and why LOCAL_QUORUM is the production default

Replication to another data centre is asynchronous and continuous. Each DC has its own replication factor, so a keyspace can be RF 3 in two regions and RF 1 in a third used only for analytics.

LOCAL_QUORUM requires a quorum within the client's own data centre only. It gives consistency locally without paying cross-region latency on every operation, and keeps working when the link between regions is down. This is what almost every production cluster uses.

EACH_QUORUM requires a quorum in every data centre — across regions, and available the moment one region is unavailable.

Two corrections worth stating explicitly, because reference material gets both wrong.

1. **RF** is not "one replica". That describes ONE. **ANY** succeeds when every replica is down, by storing a hint — the write is durable but not readable until replay.

2. **QUORUM** is not the default. The default is ONE, a cluster where nobody set a consistency level is running at ONE, not at a balanced middle setting.

Apply R + W - RF to choose. With RF 3, the common production pairing is write LOCAL_QUORUM and read LOCAL_QUORUM — 2 + 2 - 3, consistent within the region, and tolerant of one node down. Drop either side to ONE and you have chosen speed over the guarantee, which is legitimate for metrics and logs and wrong for anything a user reads back immediately.

How divergence is repaired — three mechanisms

Hinted handoff

The coordinator spares writes for a briefly unavailable node and replays them. Bounded by a time window — beyond it, hints are dropped

Read repair

Divergence found during a read is corrected on the spot. Only helps data that is actually read

Anti-entropy repair

node1tool repair. The only mechanism that reaches every row. Not optional — see panel 9

Conflicts resolve by last-write-wins, per column, on timestamp. There is no merge and no conflict surface. Clock skew across nodes therefore causes silent data loss — a write with an earlier timestamp is discarded. Run NTP everywhere and monitor drift; this is not a theoretical concern.

The heap guidance. Keep the JVM heap moderate — commonly 8 to 16 GB with the modern collectors — and leave the rest of RAM to the page cache. Long garbage-collection pauses cause nodes to be marked down by their peers, which triggers hints, repairs and cascading load. Pause time matters more than heap size, and it is the metric to alert on.

Read amplification, and why it is the thing to watch

A single logical row can have fragments in the memtable and in many SSTables — one holding the original insert, another an updated column, another a tombstone. Every read must consult all of them and merge the touches.

So the number of SSTables a read bounces is the read cost. Compaction exists to keep that number low. When compaction falls behind — because the strategy is wrong, or the node is IO saturated — read latency degrades steadily with no change to the queries. The metric to watch is SSTable per read, and it is one of the most diagnostic numbers in the whole system.

- Compression is on by default — LZ4 is fast and usually correct. Chunk length trades ratio against read speed, so smaller chunks suit random reads
- Local storage, not network-attached. Cassandra assumes fast local disks; putting SSTables on remote storage undermines the design
- Leave read headroom. Compaction needs space to merge output before removing inputs — a disk near capacity cannot compact, which increases read cost and fills the disk further
- Never use RAID with parity for data. Cassandra replicates across nodes already; disk-level redundancy adds write cost for a guarantee you have elsewhere

The heap guidance. Keep the JVM heap moderate — commonly 8 to 16 GB with the modern collectors — and leave the rest of RAM to the page cache. Long garbage-collection pauses cause nodes to be marked down by their peers, which triggers hints, repairs and cascading load. Pause time matters more than heap size, and it is the metric to alert on.

10 · TRANSACTIONS & BATCHES

Lightweight transactions

```
INSERT INTO users (email, id) VALUES (?, ?) IF NOT EXISTS;
```

UPDATE accounts SET balance = ? WHERE id = ? IF balance < ?; -- compare and set

What they are, precisely. Linearizable compare-and-set within a single partition, implemented with Paxos. They are not multi-row transactions, not multi-table, and there is no rollback. Two statements with IF are two independent transactions.

And they are expensive. Paxos requires several round trips among replicas rather than one — commonly quoted as roughly four — so an LWT can cost an order of magnitude more than a normal write. Contention on the same partition makes it worse, because rounds fail and retry.

Use them for genuine uniqueness and state-transition guards — claiming a username, moving an order from pending to paid exactly once. Do not use them on a hot path, and never as a general substitute for the transactions a relational database provides. If a design needs them frequently, Cassandra is likely the wrong choice for that part of the system.

Batches — the most misunderstood feature here

A batch is not a performance optimization. It usually makes things slower. A logged batch spanning several partitions gives atomicity — all statements eventually apply — at the cost of writing a batch to two additional nodes first, and forcing one coordinator to fan out to many replicas it does not own. The parallelism of independent writes is lost.

Single-partition batch

Genuinely useful. All statements go to one set of replicas and are applied atomically and isolated. This is the good case

Multi-partition logged batch

Use only when atomicity across partitions is a real requirement — for example keeping denormalised copies in step. Accept the cost knowingly

Multi-partition unlogged batch

Almost always wrong. No atomicity and coordinator overhead. Send consistent individual writes instead

The rule: batch for atomicity, never for throughput. For throughput, issue many asynchronous single-row writes — that is what the architecture is built for, and it will be faster.

Dropped messages in tp1stats are the clearest outward signal there is. They mean the node could not process requests within their timeout and discarded them — silently, from the client's point of view, at low consistency levels. Any non-zero dropped count deserves investigation.

Query-first modelling, tunable consistency, and the two things that will actually break it

data modelling · consistency · write & read path · compaction · tombstones & repair · operations · depth marked junior — staff

3 · ANTI-PATTERNS — the ones that reliably cause incidents

Do not use it when

- Queries are ad-hoc or exploratory. You must know them before designing the table — this is the hardest constraint to accept and the most commonly underestimated
- The data is relational with joins across entities — there are no joins, and doing them in the application at scale is painful
- You need multi-row or multi-table ACID transactions — see the note below
- The workload is delete-heavy, or resembles a queue. Tombstones make this actively hostile
- The dataset is small. A three-node minimum for meaningful replication is overkill for data that fits on one machine
- Aggregations across partitions are needed — that is an analytics engine's job

The correction worth making early: lightweight transactions are not ACID transactions. An IF clause gives linearizable compare-and-set within a single partition, implemented with Paxos. It is not multi-row, not multi-table, has no rollback, and costs several round trips. Describing it as ACID sets an expectation the database cannot meet.

STAFF Version control. Cassandra 5.0 brought the most consequential changes in years: Storage-Attached Indexes, which fix much of what made secondary indexes unusable; a vector type; approximate-nearest-neighbour search comparable with normal CQL filters; the Unified Compaction Strategy, which subsumes the older strategies and self-tunes; the biased memtable and the BT1 SSTable format; and dynamic data masking. If you last evaluated Cassandra on 3.x, several conclusions are now out of date.

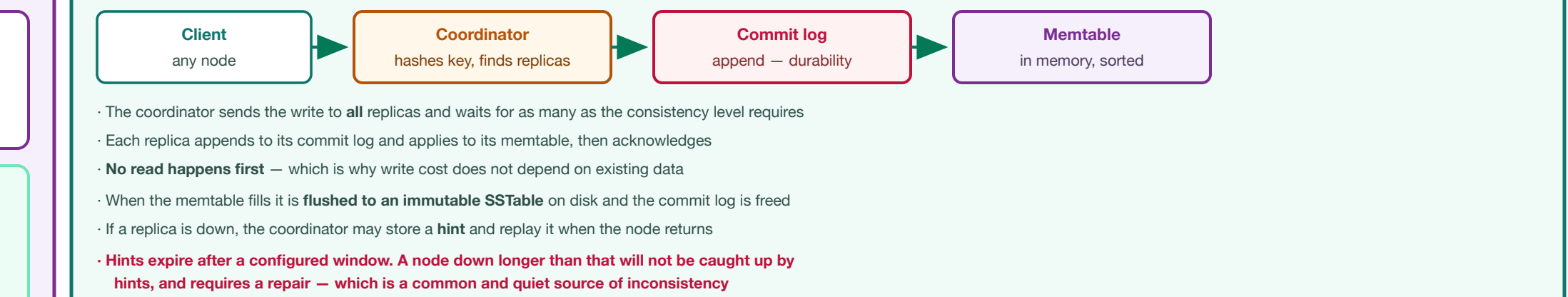
3 · ANTI-PATTERNS

Anti-pattern	Why it fails
Using it as a queue	The classic insert, read the oldest, delete, repeat — every delete leaves a tombstone at the front of the partition, so each read scans through more and more of them until it times out. Cassandra is actively bad at this, and no tooling fixes it.
Delete-heavy workloads	Same mechanism. Deletes are writes, and the space is only reclaimed after compaction and the grace period. Prefer TTLs, or partition to removal is dropping whole partitions
Read-modify-write	Read a value, modify it, write it back. Two concurrent clients silently lose one update — there is no locking. Use a lightweight transaction if you truly need it, and expect the cost
ALLOW FILTERING	It does not enable a feature — it disables a safety check and permits a scan across nodes. Fine in an ad-hoc investigation, never in application code
Cross-partition batches	Treated as a performance optimisation when it is the opposite. See panel 10
Secondary indexes on high-cardinality columns	Historically fans out to every node. See panel 11
Unbounded collections	A list or map that keeps growing is read and rewritten whole. Keep them small and bounded, or use clustering rows instead
One giant table for everything	Relational instinct. One table per query is the model

The common thread. Every one of these is a relational or queue-shaped habit applied to a store that is optimised for a different access pattern. They usually work in development, where volumes are small and partitions are short, and fail mercilessly when in production once the data has accumulated.

The counter-question that catches most of them: "which partition does this query read, and how large will it be in a year?" If the answer is "several" or "we don't know", the design needs another pass before it ships.

THE WRITE PATH — why writes are fast and uniform



THE READ PATH — more work than a write, and the asymmetry is the point

A row may have fragments in the memtable and in several SSTables, each holding a different version of different columns. A read must find and merge them all, then merge conflicts to consistency — last-write-wins, per column.

Bloom filter per SSTable answers "definitely not here" cheaply, so most SSTables are skipped without touching disk.

Key cache and the partition index locate the row within an SSTable that has the column it.

Read repair compares replicas and pushes corrections back, helping divergence as a side effect of reading.

This is why compaction matters so much: the more SSTables a partition is spread across, the more work every read does.

6 · COL & QUERY RULES

CQL looks like SQL and is not SQL. There are no joins, no subqueries, no aggregates across partitions and no query planner. The syntax being familiar is the trap — what a query is allowed to do is determined entirely by the primary key of the table it reads.

What a WHERE clause may contain

- The full partition key is required — always, with equality. Without it the cluster cannot know which nodes to ask
- Columns live in their own tables and cannot mix with normal columns. They are not denormalised, so a hinted increment may double-count
- Only the last restricted clustering column may use a range. Everything before must be equality
- ORDER BY can only follow the clustering order, forwards or reversed
- Anything else requires ALLOW FILTERING, which is a warning rather than a feature

```
-- PRIMARY KEY (user_id, event_time, event_id)
OK WHERE user_id = ?
OK WHERE user_id = ? AND event_time > ?
OK WHERE user_id = ? AND event_time = ? AND event_id = ?
NO WHERE event_time > ? -- no partition key
NO WHERE user_id = ? AND event_id = ? -- skips a clustering cell
```

These restrictions are a feature, not a limitation. The database refuses queries it cannot serve efficiently, rather than accepting them and degrading unpredictably at scale — which is what a relational database would do. ALLOW FILTERING switches that protection off, and it is the reason it should never appear in application code.

Other things worth knowing

- TTL, per row or row column — data expires automatically. The best answer to deletion, since it avoids explicit tombstones from application logic
- Counters live in their own tables and cannot mix with normal columns. They are not denormalised, so a hinted increment may double-count
- Collections — set, list and map. Keep them small; a non-frozen collection is read and rewritten as a unit
- INSERT and UPDATE are the same operation — both are updates. There is no "row does not exist" error
- Paging is automatic in the drivers. Do not implement it manually with offsets

8 · COMPACTION

Compaction merges SSTables, discarding superseded values and expired tombstones. It is what keeps reads fast and what reclaiming disk. Choosing the wrong strategy is one of the most consequential and most reversible decisions here.

Strategy	Behavior and fit
STCS	Merges SSTables of similar size. Low write amplification, higher read amplification, and can need up to double the space during a large compaction. Suits write-heavy workloads with few overwrites
LCS	Maintains levels so a row appears in few SSTables. Excellent read performance and much higher write amplification. Suits read-heavy workloads with frequent updates — and will struggle if the write rate exceeds what it can compact
TWCS	The correct answer for time-series with TTLs. Groups data by time window so an entire window expires and is dropped as a unit, with no tombstones scanning at all
UCS	Subsumes the tiered and levelled behaviours behind one configurable strategy that adapts as the workload changes. The direction of travel, and it removes much of the guesswork above

The decision, in one line each. Time-series with a TTL — TWCS, and the benefit is large. Read-heavy with updates — LCS. Write-heavy, few updates — STCS. On 5.0, UCS unless you have a specific reason.